

JAVA BST CODE

1. Node + Insert

```
class Node{
    int data;
    Node left,right;
    Node(int x){
        data=x;
        left=right=null;
    }
}

class BST{
    Node insert(Node root,int x){
        if(root==null) return new Node(x);
        if(x<root.data) root.left=insert(root.left,x);
        else root.right=insert(root.right,x);
        return root;
    }
}
```

2. Traversals

```
void inorder(Node root){
    if(root==null) return;
    inorder(root.left);
    System.out.print(root.data+" ");
    inorder(root.right);
}
```

```
void preorder(Node root){
    if(root==null) return;
    System.out.print(root.data+" ");
    preorder(root.left);
    preorder(root.right);
}
```

```
void postorder(Node root){
    if(root==null) return;
    postorder(root.left);
    postorder(root.right);
    System.out.print(root.data+" ");
}
```

3. Search

```
Node search(Node root,int key){
    if(root==null || root.data==key) return root;
    if(key<root.data) return search(root.left);
    return search(root.right);
}
```

4. Delete

```
Node minValue(Node root){
    while(root.left!=null) root=root.left;
    return root;
}
```

```
Node deleteNode(Node root,int key){
```

```
if(root==null) return root;
if(key<root.data) root.left=deleteNode(root.left,key);
else if(key>root.data) root.right=deleteNode(root.right,key);
else{
    if(root.left==null) return root.right;
    else if(root.right==null) return root.left;
    Node temp=minValue(root.right);
    root.data=temp.data;
    root.right=deleteNode(root.right,temp.data);
}
return root;
}
```